

Strukturalni paterni

Adapter patern

U postojećem sistemu postoji klasa *Dobavljac* koja ima metodu *posaljiPonudu()*, dok klasa *NabavkaKnjiga* koristi dobavljača za kreiranje narudžbe. Međutim, u realnom okruženju biblioteka može sarađivati sa više različitih dobavljača, pri čemu svaki od njih može imati drugačiji način komunikacije i različite nazive metoda za obavljanje istih operacija. Ovaj problem dovodi do toga da bi klasa *NabavkaKnjiga* morala sadržavati dodatnu logiku za prilagođavanje svakom pojedinačnom dobavljaču, što bi povećalo složenost sistema i narušilo njegovu fleksibilnost. Svako uvođenje novog dobavljača zahtijevalo bi izmjene u postojećem kodu, što nije poželjno.

Uvođenjem Adapter paterna definiše se jedinstveni interfejs za komunikaciju sa dobavljačima, dok se za svakog konkretnog dobavljača implementira adapter koji prevodi taj standardizovani poziv u odgovarajući oblik koji taj dobavljač podržava. Na ovaj način klasa *NabavkaKnjiga* komunicira isključivo preko jedinstvenog interfejsa, bez potrebe da zna detalje implementacije svakog dobavljača. Time se postiže lakše održavanje sistema i jednostavnije proširenje novim dobavljačima bez izmjene postojećeg koda.

Decorator patern

Klasa *Knjiga* sadrži osnovne podatke i funkcionalnosti vezane za knjigu, kao što su provjera dostupnosti i pretraga. Međutim, u praksi se često javlja potreba za dodatnim funkcionalnostima, kao što su označavanje knjige kao preporučene, praćenje popularnosti ili dodavanje dodatnih informacija za prikaz korisnicima.

Dodavanje svih tih funkcionalnosti direktno u klasu *Knjiga* dovelo bi do njenog preopterećenja i smanjena preglednosti, čime bi se narušio princip jedne odgovornosti (SRP). Uvođenjem Decorator paterna omogućava dinamičko proširenje funkcionalnosti objekta *Knjiga* bez izmjene njegove osnovne strukture. Dodatne funkcionalnosti se implementiraju kroz posebne dekoratorske klase koje omotavaju osnovni objekat i proširuju njegovo ponašanje. Na ovaj način sistem postaje fleksibilniji, jer se funkcionalnosti mogu dodavati ili kombinovati po potrebi, bez uticaja na postojeći kod.

Facade patern

Sistem biblioteke sastoji se od više međusobno povezanih klasa, kao što su *ZahtjevZaPosudbu*, *Posudba*, *ClanskaKartica*, *KorisnikSistema* i *Knjiga*. Za izvršavanje jedne složene operacije, poput posudbe knjige, potrebno je izvršiti više koraka: provjeriti validnost članstva, provjeriti dostupnost knjige, kreirati zahtjev, odobriti zahtjev i na kraju evidentirati posudbu. Ovakav način uvodi kompleksnost za klijentski dio sistema, jer mora direktno komunicirati sa više klasa i upravljati njihovom međusobnom saradnjom.

Uvođenjem Facade paterna definiše se jedinstvena klasa koja predstavlja pojednostavljeni interfejs prema složenom podsistemu. Ta klasa objedinjuje sve potrebne operacije i omogućava klijentu da izvrši kompleksnu funkcionalnost putem jednog poziva. Time se smanjuje kompleksnost korištenja sistema, poboljšava preglednost i smanjuje zavisnost između klasa.

Composite patern

U klasi *Knjiga* atribut *kategorija* trenutno je predstavljen kao jednostavna vrijednost *String*. Međutim, u realnim sistemima kategorije često imaju hijerarhijsku strukturu. Jednostavna reprezentacija kategorije ne omogućava rad sa složenijim strukturama, niti fleksibilno organizovanje knjiga.

Uvođenjem Composite paterna omogućava se modeliranje hijerarhije kategorija kroz jedinstvenu strukturu u kojoj se i pojedinačni i složeni elementi tretiraju na isti način. Na taj način moguće je formirati stablo kategorija gdje svaka kategorija može sadržavati podkategorije. Ovaj pristup omogućava jednostavnije upravljanje kategorijama i fleksibilnije proširenje sistema bez izmjene postojećih dijelova.

Proxy patern

Klasa *Knjiga* sadrži podatke poput *naslovnica*, koji mogu predstavljati reursno zahtjevne objekte (npr. slike velike rezolucije). Učitavanje svih tih podataka unaprijed može negativno utjecati na performanse sistema, posebno ako se radi o velikom broju knjiga. Bez dodatne kontrole, sistem bi učitavao sve podatke odmah, bez obzira na to da li su zaista potrebni korisniku.

Uvođenjem Proxy paternu uvodi se zamjenski objekat koji kontroliše pristup stvarnom objektu. Konkretno, naslovnica knjige može se učitati tek u trenutku kada je zaista potrebna. Na ovaj način poboljšavaju se performanse sistema i optimizuje korištenje resursa, jer se skupi objekti kreiraju samo kada su neophodni.

Bridge patern

Klasa *KorisnikSistema* u sebi sadrži atribut *uloga* (*clan*, *bibliotekar*, *administrator*), što može dovesti do potrebe za različitim ponašanjima u zavisnosti od uloge korisnika. Kako sistem raste, može doći do povećanja broja kombinacija između tipova korisnika i njihovih funkcionalnosti. Direktno implementiranje svih tih varijacija unutar jedne klase dovelo bi do složenog i teško održivog koda.

Uvođenjem Bridge paternu razdvaja se apstrakcija korisnika od implementacije njegove uloge. Na taj način se ponašanja vezana za uloge implementiraju nezavisno od same strukture korisnika. Ovaj pristup omogućava lakše proširenje sistema novim ulogama ili funkcionalnostima bez izmjene postojećeg koda, čime se povećava fleksibilnost i održivost sistema.